Practical 11

File I/O, Serialization & NIO

Part A — Practice Programs (Hour 1)

1) Save/Load — ObjectOutputStream, ObjectInputStream and a transient field

Code

```
import java.io.*;

public class SaveLoad {
    static class Student implements Serializable {
        private static final long serialVersionUID = 1L;
        String name;
        int marks;
        transient String password;      // transient: not written to the file

        Student(String name, int marks, String password) {
            this.name = name;
            this.marks = marks;
            this.password = password;
        }

        public String toString() {
            return name + " | marks=" + marks + " | password=" + password;
        }
    }

    public static void main(String[] args) throws Exception {
        Student[] students = {
            new Student("Riya", 88, "secret1"),
            new Student("Aman", 74, "secret2"),
            new Student("Neha", 91, "secret3")
        };
        System.out.println("Before saving:");
        for (Student s : students) System.out.println("  " + s);

        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("students.dat")))
            {
            out.writeObject(students);
        }

        Student[] loaded;
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("students.dat"))) {
            loaded = (Student[]) in.readObject();
        }
        System.out.println("After loading into fresh objects:");
        for (Student s : loaded) System.out.println("  " + s);
        System.out.println("Same objects? " + (loaded[0] == students[0]) + " | Same values? " +
            (loaded[0].name.equals(students[0].name)
                && loaded[0].marks == students[0].marks));
    }
}
```

Output

```
Before saving:
  Riya | marks=88 | password=secret1
  Aman | marks=74 | password=secret2
  Neha | marks=91 | password=secret3
After loading into fresh objects:
  Riya | marks=88 | password=null
  Aman | marks=74 | password=null
  Neha | marks=91 | password=null
Same objects? false | Same values? true
```

2) Log Analyzer — Path/Files, line counting and file attributes

Code

```
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.List;

public class LogAnalyzer {
    public static void main(String[] args) throws IOException {
        Path dir = Path.of("logs_demo");
        Files.createDirectories(dir);
        Files.write(dir.resolve("app1.log"), List.of("INFO start", "ERROR disk full", "INFO
            retry"));
        Files.write(dir.resolve("app2.log"), List.of("INFO start", "INFO ok"));
        Files.write(dir.resolve("app3.log"), List.of("ERROR timeout", "ERROR timeout again", "INFO
            end", "INFO bye"));

        String keyword = "ERROR";
        int totalLines = 0, matching = 0;
        try (DirectoryStream<Path> files = Files.newDirectoryStream(dir, "*.log")) {
            for (Path f : files) {
                List<String> lines = Files.readAllLines(f);
                int m = 0;
                for (String line : lines) if (line.contains(keyword)) m++;
                BasicFileAttributes attrs = Files.readAttributes(f, BasicFileAttributes.class);
                System.out.println(f.getFileName() + ": " + lines.size() + " lines, " + m + " with
                    " + keyword + ", size " + attrs.size() + " bytes");
                totalLines += lines.size();
                matching += m;
            }
        }
        System.out.println("Total lines: " + totalLines + ", lines containing " + keyword + ": " +
            matching);
    }
}
```

Output

```
app2.log: 2 lines, 0 with ERROR, size 19 bytes
app3.log: 4 lines, 2 with ERROR, size 52 bytes
app1.log: 3 lines, 1 with ERROR, size 38 bytes
Total lines: 9, lines containing ERROR: 3
```

Directory streams return files in no guaranteed order.

3) Folder Report (For Advanced Learners — home) — walkFileTree and summary file

Code

```
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.ArrayList;
import java.util.List;

public class FolderReport {
    public static void main(String[] args) throws IOException {
        Path root = Path.of("tree_demo");
        Files.createDirectories(root.resolve("docs/notes"));
        Files.writeString(root.resolve("readme.txt"), "hello world\n");
        Files.writeString(root.resolve("docs/guide.md"), "# Guide\nSome text here\n");
        Files.writeString(root.resolve("docs/notes/todo.txt"), "buy milk\n");

        List<String> report = new ArrayList<>();
        report.add(String.format("%-28s %8s  %s", "File", "Bytes", "Modified"));
        long[] total = {0, 0};
        Files.walkFileTree(root, new SimpleFileVisitor<Path>() {
            @Override
            public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) {
                report.add(String.format("%-28s %8d  %s", root.relativize(file), attrs.size(),
                    attrs.lastModifiedTime()));
                total[0]++;
                total[1] += attrs.size();
                return FileVisitResult.CONTINUE;
            }
        });
        report.add("Files: " + total[0] + ", total size: " + total[1] + " bytes");
        Files.write(Path.of("tree_report.txt"), report);
        System.out.println("Report written to tree_report.txt:");
        report.forEach(System.out::println);
    }
}
```

Output

```
Report written to tree_report.txt:
File                           Bytes  Modified
readme.txt                        12  2026-10-20T18:11:26.597245076Z
docs/guide.md                     23  2026-10-20T18:11:26.597245076Z
docs/notes/todo.txt                9  2026-10-20T18:11:26.597245076Z
Files: 3, total size: 44 bytes
```

The report lists name, size and modified time for every file and is also written to tree_report.txt.

Part B — Project Work (Hour 2): MiniBank

MiniBank now remembers its data: the model is Serializable, StatePersister saves and loads all accounts, TransactionLog appends to a log file, and ReportGenerator walks the logs folder with NIO to build an end-of-day report. Files not listed are unchanged from Practical 10.

1) Account (Serializable, serialVersionUID, transient withdrawnToday) and Customer

Code

model/Account.java

```
package model;

import exception.BankException;
import exception.DailyLimitExceededException;
import exception.InsufficientFundsException;
import exception.InvalidAmountException;
import model.annotation.Id;
import model.annotation.MaxLength;
import model.annotation.Positive;

import java.io.Serializable;
import java.util.Objects;

public class Account implements Transactable, InterestBearing, Serializable {
    private static final long serialVersionUID = 1L;
    public static final long DAILY_LIMIT = 10000;

    @Id
    @MaxLength(10)
    protected String accountNumber;
    @Positive
    protected long balance;
    private double rate;
    private transient long withdrawnToday;   // transient: NOT saved, resets to 0 after loading

    public Account(String accountNumber, long initialBalance, double rate) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
        this.rate = rate;
    }

    // Thread-safe: only one thread at a time may run a synchronized method of this object
    @Override
    public synchronized void deposit(long amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
```

```
        }
        balance += amount;
    }

    @Override
    public synchronized boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException, DailyLimitExceededException {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
        }
        if (withdrawnToday + amount > DAILY_LIMIT) {
            throw new DailyLimitExceededException(DAILY_LIMIT);
        }
        if (amount > balance) {
            throw new InsufficientFundsException(amount - balance);
        }
        balance -= amount;
        withdrawnToday += amount;
        return true;
    }

    public void transfer(Account to, long amount) throws BankException {
        boolean done = false;
        try {
            withdraw(amount);
            to.deposit(amount);
            done = true;
        } catch (BankException e) {
            System.out.println("Transfer failed: " + e.getMessage());
            throw e;
        } finally {
            System.out.println("Transfer attempt " + accountNumber + " -> "
                    + to.getAccountNumber() + " finished, success=" + done);
        }
    }
    @Override
    public double interestRate() {
        return rate;
    }

    @Override
    public long getBalance() {
        return balance;
    }

    public long getWithdrawnToday() {
        return withdrawnToday;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    @Override
    public String toString() {
        return "Account[" + accountNumber + ", balance=" + balance + "]";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (!(o instanceof Account)) {
            return false;
        }
        Account other = (Account) o;
        return accountNumber.equals(other.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

model/Customer.java

```
package model;

import java.io.Serializable;

public class Customer implements Serializable {
    private static final long serialVersionUID = 1L;

    private final String customerId;
    private String name;
    private String email;

    public Customer(String customerId, String name, String email) {
        this.customerId = customerId;
        this.name = name;
        this.email = email;
    }

    public String getCustomerId() { return customerId; }
    public String getName() { return name; }
    public String getEmail() { return email; }

    @Override
    public String toString() {
        return "Customer[" + customerId + ", " + name + ", " + email + "]";
    }
}
```

PremiumAccount only adds its own serialVersionUID. Supplementary problem covered: withdrawnToday is transient, so it is not saved.

2) StatePersister — ObjectOutputStream / ObjectInputStream in try-with-resources

Code

service/StatePersister.java

```
package service;

import model.Account;

import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.nio.file.Files;
import java.nio.file.Path;

public class StatePersister {
    public static void save(Account[] accounts, Path file) throws IOException {
        if (file.getParent() != null) {
            Files.createDirectories(file.getParent());
        }
        try (ObjectOutputStream out = new ObjectOutputStream(Files.newOutputStream(file))) {
            out.writeObject(accounts);
        }
    }

    public static Account[] load(Path file) throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(Files.newInputStream(file))) {
            return (Account[]) in.readObject();
        }
    }
}
```

3) TransactionLog — Files.write with CREATE and APPEND

Code

service/TransactionLog.java

```
package service;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;

public class TransactionLog {
    public static synchronized void append(Path file, String line) throws IOException {
        if (file.getParent() != null) {
            Files.createDirectories(file.getParent());
        }
        Files.write(file, (line + System.lineSeparator()).getBytes(StandardCharsets.UTF_8),
                StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    }
}
```

4) ReportGenerator — newDirectoryStream, BufferedReader, readAttributes

Code

service/ReportGenerator.java

```
package service;

import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.DirectoryStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.ArrayList;
import java.util.List;

public class ReportGenerator {
    public static String generate(Path logsDir, Path reportFile) throws IOException {
        long deposits = 0, withdrawals = 0;
        List<String> lines = new ArrayList<>();
        lines.add("===== End-of-day report =====");

        try (DirectoryStream<Path> stream = Files.newDirectoryStream(logsDir)) {
            for (Path file : stream) {
                BasicFileAttributes attrs = Files.readAttributes(file, BasicFileAttributes.class);
                // supplementary: skip empty files and anything that is not a .log file
                if (attrs.size() == 0 || !file.getFileName().toString().endsWith(".log")) {
                    lines.add("Skipped " + file.getFileName());
                    continue;
                }
                lines.add("File " + file.getFileName() + " | size " + attrs.size() + " bytes |
                    modified " + attrs.lastModifiedTime());
                try (BufferedReader reader = Files.newBufferedReader(file)) {
                    String line;
                    while ((line = reader.readLine()) != null) {
                        String[] parts = line.trim().split("\\s+");
                        if (parts.length != 3) continue;
                        long amount = Long.parseLong(parts[2]);
                        if (parts[0].equals("DEPOSIT")) deposits += amount;
                        else if (parts[0].equals("WITHDRAW")) withdrawals += amount;
                    }

                }
            }
        }
        lines.add("Total deposits    : " + deposits);
        lines.add("Total withdrawals : " + withdrawals);
        lines.add("Net change        : " + (deposits - withdrawals));
        Files.write(reportFile, lines);
        return String.join(System.lineSeparator(), lines);
    }
}
```

Supplementary problem covered: empty files and files that do not end with .log are skipped.

5) main — save, reload, print, then generate the report

Code

service/MiniBank.java

```
package service;

import exception.BankException;
import model.Account;
import model.PremiumAccount;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Comparator;

public class MiniBank {
    public static void main(String[] args) throws Exception {
        Path dataDir = Path.of("data");
        Path logsDir = dataDir.resolve("logs");
        // start from a clean data folder so the demo is repeatable
        if (Files.exists(dataDir)) {
            try (var walk = Files.walk(dataDir)) {
                walk.sorted(Comparator.reverseOrder()).forEach(p -> p.toFile().delete());
            }
        }
        Files.createDirectories(logsDir);

        Account[] accounts = {
            new Account("AC0001", 5000, 0.04),
            new Account("AC0002", 2000, 0.04),
            new PremiumAccount("AC0003", 9000, 0.06)
        };

        // transactions are applied AND logged
        Path log = logsDir.resolve("2026-10-20.log");
        try {
            accounts[0].deposit(500);
            TransactionLog.append(log, "DEPOSIT AC0001 500");
            accounts[1].withdraw(200);
            TransactionLog.append(log, "WITHDRAW AC0002 200");

            accounts[2].deposit(1000);
            TransactionLog.append(log, "DEPOSIT AC0003 1000");
            accounts[2].withdraw(300);
            TransactionLog.append(log, "WITHDRAW AC0003 300");
        } catch (BankException e) {
            System.out.println("Error: " + e.getMessage());
        }
        // extra files to show filtering
        Files.writeString(logsDir.resolve("notes.txt"), "not a log\n");
        Files.writeString(logsDir.resolve("empty.log"), "");

        System.out.println("--- Before save (withdrawnToday of AC0002 = " +
            accounts[1].getWithdrawnToday() + ") ---");
        for (Account a : accounts) System.out.println(a);

        Path file = dataDir.resolve("accounts.dat");
        StatePersister.save(accounts, file);
        System.out.println("Saved " + accounts.length + " accounts to " + file + " (" +
            Files.size(file) + " bytes)");

        Account[] loaded = StatePersister.load(file);
        System.out.println("--- After load into a new array ---");
        for (Account a : loaded) System.out.println(a + (a instanceof PremiumAccount ? "
            (PremiumAccount preserved)" : ""));
        System.out.println("transient field withdrawnToday of AC0002 after load = " +
            loaded[1].getWithdrawnToday() + " (was not saved)");
        System.out.println("Loaded equals saved: " + loaded[0].equals(accounts[0]) + ", balance
            match: "
                + (loaded[0].getBalance() == accounts[0].getBalance()));

        System.out.println("--- Report ---");
        System.out.println(ReportGenerator.generate(logsDir, dataDir.resolve("report.txt")));
    }
}
```

Output

```
--- Before save (withdrawnToday of AC0002 = 200) ---
Account[AC0001, balance=5500]
Account[AC0002, balance=1800]
Account[AC0003, balance=9700]
Saved 3 accounts to data/accounts.dat (247 bytes)
--- After load into a new array ---
Account[AC0001, balance=5500]
Account[AC0002, balance=1800]
Account[AC0003, balance=9700] (PremiumAccount preserved)
transient field withdrawnToday of AC0002 after load = 0 (was not saved)
Loaded equals saved: true, balance match: true
--- Report ---
===== End-of-day report =====
Skipped empty.log
File 2026-10-20.log | size 79 bytes | modified 2026-10-20T18:11:27.745245024Z
Skipped notes.txt
Total deposits    : 1500
Total withdrawals : 500
Net change        : 1000
```

Package layout, build and tag

Code

```
minibank/
 |-- exception/
 |   |-- AccountNotFoundException.java
 |   |-- BankException.java
 |   |-- DailyLimitExceededException.java
 |   |-- InsufficientFundsException.java
 |   +-- InvalidAmountException.java
 |-- model/
 |   |-- annotation/
 |   |   |-- Id.java
 |   |   |-- MaxLength.java
 |   |   +-- Positive.java
 |   |-- Account.java
 |   |-- Customer.java
 |   |-- InterestBearing.java
 |   |-- Premium.java
 |   |-- PremiumAccount.java
 |   +-- Transactable.java
 |-- service/
 |   |-- MiniBank.java
 |   |-- ReportGenerator.java
 |   |-- StatePersister.java
 |   |-- TransactionLog.java
 |   |-- TransactionProcessor.java
 |   +-- WithdrawRule.java
 +-- util/
     |-- AnnotationValidator.java
     +-- Validator.java

javac -d out $(find exception model util service -name "*.java")
java -cp out service.MiniBank
cd out
jar --create --file minibank.jar --main-class=service.MiniBank .
java -jar minibank.jar
cd ..
echo "data/" >> .gitignore      # do not commit generated data files

git add .
git commit -m "Practical 11 - serialization and NIO reports"
git tag lab-11
```

The reloaded accounts print with the same numbers and balances as the saved ones; the report shows total deposits 1500, total withdrawals 500 and net change 1000, which matches the four logged transactions.

Key Questions / Analysis / Interpretation

1. What does it mean for a class to be Serializable, and why add serialVersionUID?

Implementing the marker interface Serializable allows an object's state to be converted to a byte stream and rebuilt later. serialVersionUID is a version number for the class: when loading, Java compares the stored value with the class's value and throws InvalidClassException if they differ. Declaring it explicitly stops harmless changes to the class from breaking old saved files.

2. What is the difference between a byte stream and a character stream?

Byte streams (InputStream/OutputStream, ObjectOutputStream) read and write raw 8-bit bytes and suit binary data such as serialized objects. Character streams (Reader/Writer, BufferedReader) read and write 16-bit characters

and handle text encoding, so they are used for text files such as the transaction log.

3. How does NIO's Path/Files API differ from the old File class, and how is a directory stream iterated?

Path is an immutable path object and Files has static methods (readAllLines, write, walk, readAttributes) that throw useful exceptions, support attributes, symbolic links and copy/move options. The old File class gives limited error information (many methods just return false). A DirectoryStream<Path> from Files.newDirectoryStream(dir) is iterated with a for-each loop inside try-with-resources so it is closed afterwards.